

DAY 3

Module 8

Common Pitfalls and Anti-Patterns

Most anti-patterns are earlier shortcuts catching up with you.

What You'll Learn

1

Recognize over-specifying and under-specifying and find the right level of detail

2

Weigh token and cost tradeoffs against rework avoided

3

Learn from documented, framework-specific pitfalls in both tools

Over- and Under-Specifying

Too much detail in a spec or plan slows everyone down and gets stale fast; too little pushes ambiguity downstream where it's more expensive to fix.

A thin constitution or a skipped clarify pass usually shows up here first: problems from earlier modules resurface as anti-patterns later.

Token and Cost Tradeoffs

More tokens spent up front on a clear spec and plan is typically offset by far fewer rework cycles.

Treat this as a deliberate tradeoff to manage, not a cost to minimize blindly.

WHY IT MATTERS

Most anti-patterns are visible in hindsight, rarely in the moment.

The team that skips a gate almost never notices until the cost shows up downstream.

Treat every skipped step as a deliberate risk decision, not a shortcut.

APPLYING THIS WITH

GitHub Spec Kit

Where Teams Get Stuck

Stale checklist: Treating a passed checklist as permanently done rather than re-running it after changes: a checklist reflects the spec at the moment it was generated, and later edits can silently invalidate it.

Ignoring analyze: Ignoring `/speckit.analyze` findings instead of returning to the step that owns them (specify or clarify for requirement problems, plan for design problems) lets small inconsistencies compound.

APPLYING THIS WITH

OpenSpec

Documented Failure Modes

From OpenSpec's docs: "Change not found," "No artifacts ready," and "Artifacts not generating properly" are named directly in OpenSpec's own troubleshooting guide. Thin project context in config.yaml is a common root cause.

```
/opsx:ff
```

Fast-forward generates every artifact at once. Convenient, but it skips the incremental review that `/opsx:continue` gives you, an under-specification risk if nobody reads each artifact before the next is generated.

Module 8 at a Glance

	GitHub Spec Kit	OpenSpec
Common pitfall	Stale checklists or ignored analyze findings	Over-using /opsx:ff without review
Root cause	A review step was skipped	A review step was skipped
Documented by	Workflow reference (checklist, analyze)	OpenSpec's own troubleshooting guide

Key Takeaways

- Anti-patterns are rarely new mistakes; they're earlier shortcuts catching up with you.
- Every framework gate you skip is a bet that nothing needed catching.
- Case studies at scale tend to share one pattern: the team that skipped a gate is the team that got surprised later.